

Servers and Routes

Section Review

Clients and Servers

- A **server** is a continuously running program that responds to requests from **clients**.
- In local development, our computer acts as both client and server.
- In production, the FastAPI server is hosted on a cloud platform.

□ Create a FastAPI Application

- Import the **FastAPI** class from the **fastapi** package and instantiate it.
- Assign it to a variable (**app** is common).
- The **app** object represents the server/web application.

Defining Routes

- A **route/endpoint** is a specific path that FastAPI matches with a complementary function.
- Use **@app.get** to define a **GET** route for the given path.
- Define the route handler function below the decorator.
- The function's return value serves as the response. FastAPI will convert it to JSON.

Starting the Server

- Execute **uv run fastapi dev** to launch the FastAPI server in development mode.
- In dev mode, FastAPI will restart the server whenever a file changes.
- FastAPI uses the **uvicorn** server library under the hood.
- The server runs on **localhost** (your computer) on port 8000 by default.

Swagger Documentation

- FastAPI generates Swagger documentation for all routes at **localhost:8000/docs**.
- The docs include the HTTP methods, routes, parameters, payload shapes, and responses.
- We can also make requests to endpoints by clicking **Try it Out** and **Execute**.

Customizing Swagger Documentation

- The **FastAPI** constructor accepts **title**, **summary**, **version**, and **contact** parameters.
- The values will appear on the generated docs. These data points help when you have a publicly exposed API.

□ Status Code

- The **status_code** parameter on a route decorator sets the status code for a *success* response.
- **200 OK** is the status code for a successful **GET** request.
- The **status** module exports named constants for all HTTP status codes (**HTTP_200_OK**).